

DP-800 LAB 01

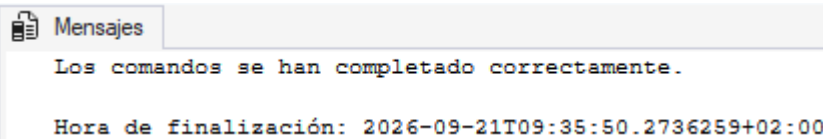


Create a new database

-Seleccionamos la carpeta de Base de Datos en SSMS y abrimos una nueva pestaña de consulta.

-Ejecutamos la siguiente consulta:

```
1 CREATE DATABASE EcommerceDB;
2 GO
3
4 USE EcommerceDB;
5 GO
```



Create core tables with constraints

-En una nueva consulta, creamos las tablas principales e insertamos datos de ejemplo.

```
1 USE EcommerceDB;
2 GO
3
4 -- Create Supplier table
5 CREATE TABLE Supplier (
6     SupplierID INT PRIMARY KEY IDENTITY(1,1),
7     SupplierName NVARCHAR(100) NOT NULL UNIQUE,
8     Country NVARCHAR(50) NOT NULL,
9     Email NVARCHAR(100),
10    Phone NVARCHAR(20),
11    CreatedDate DATETIME2 DEFAULT GETUTCDATE()
12 );
13
14 -- Create Category table
15 CREATE TABLE Category (
16     CategoryID INT PRIMARY KEY IDENTITY(1,1),
17     CategoryName NVARCHAR(100) NOT NULL UNIQUE,
18     Description NVARCHAR(500)
19 );
20
21 -- Create Product table with constraints
22 CREATE TABLE Product (
23     ProductID INT PRIMARY KEY IDENTITY(1,1),
24     ProductName NVARCHAR(100) NOT NULL,
25     CategoryID INT NOT NULL,
26     SupplierID INT NOT NULL,
27     BasePrice DECIMAL(10,2) NOT NULL,
28     StockQuantity INT NOT NULL DEFAULT 0,
29     CreatedDate DATETIME2 DEFAULT GETUTCDATE(),
30     CHECK (BasePrice > 0),
31     CHECK (StockQuantity >= 0),
32     FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID),
33     FOREIGN KEY (SupplierID) REFERENCES Supplier(SupplierID)
34 );
35
36 -- Create indexes
37 CREATE INDEX IX_Category ON Product(CategoryID);
38 CREATE INDEX IX_Supplier ON Product(SupplierID);
39
40 GO
```

Mensajes

Los comandos se han completado correctamente.

Hora de finalización: 2026-09-21T09:38:23.6891316+02:00

```

1  USE EcommerceDB;
2  GO
3
4  -- Insert sample suppliers
5  INSERT INTO Supplier (SupplierName, Country, Email, Phone)
6  VALUES
7      ('Contoso Supplies', 'USA', 'contact@contoso.com', '555-0100'),
8      ('Fabrikam Inc', 'Canada', 'sales@fabrikam.com', '555-0200');
9
10 -- Insert sample categories
11 INSERT INTO Category (CategoryName, Description)
12 VALUES
13     ('Electronics', 'Electronic devices and accessories'),
14     ('Clothing', 'Apparel and fashion items');
15
16 -- Insert sample products
17 INSERT INTO Product (ProductName, CategoryID, SupplierID, BasePrice, StockQuantity)
18 VALUES
19     ('Wireless Mouse', 1, 1, 29.99, 100),
20     ('Cotton T-Shirt', 2, 2, 19.99, 250);
21 GO

```

Create a temporal table for price history

-En una nueva consulta, creamos una tabla temporal y la consultamos.

```

1  USE EcommerceDB;
2  GO
3
4  -- Create Price History table with temporal versioning
5  CREATE TABLE ProductPrice (
6      PriceID INT PRIMARY KEY IDENTITY(1,1),
7      ProductID INT NOT NULL,
8      CurrentPrice DECIMAL(10,2) NOT NULL,
9      EffectiveDate DATE,
10     SysStartTime DATETIME2 GENERATED ALWAYS AS ROW START HIDDEN,
11     SysEndTime DATETIME2 GENERATED ALWAYS AS ROW END HIDDEN,
12     PERIOD FOR SYSTEM_TIME (SysStartTime, SysEndTime),
13     FOREIGN KEY (ProductID) REFERENCES Product (ProductID)
14 ) WITH (SYSTEM_VERSIONING = ON);
15 GO
16
17 -- Insert initial price data
18 INSERT INTO ProductPrice (ProductID, CurrentPrice, EffectiveDate)
19 VALUES (1, 99.99, '2025-01-01'), (2, 149.99, '2025-01-01');
20
21 -- Update price (creates history entry)
22 UPDATE ProductPrice SET CurrentPrice = 109.99 WHERE ProductID = 1;
23 GO

```

```

1  USE EcommerceDB;
2  GO
3
4  -- Query price history
5  SELECT ProductID, CurrentPrice, SysStartTime, SysEndTime
6  FROM ProductPrice
7  FOR SYSTEM_TIME ALL
8  WHERE ProductID = 1;

```

	ProductID	CurrentPrice	SysStartTime	SysEndTime
1	1	109.99	2026-09-21 07:47:54.7791956	9999-12-31 23:59:59.9999999
2	1	99.99	2026-09-21 07:47:54.7781940	2026-09-21 07:47:54.7791956

Add JSON columns for metadata

Creamos un campo JSON para especificar metadatos y la consultamos.

```

1  USE EcommerceDB;
2  GO
3
4  -- Add metadata column to Product (JSON type requires SQL Server 2025)
5  ALTER TABLE Product ADD Metadata JSON;
6  GO
7
8  -- Add computed column for indexing
9  ALTER TABLE Product ADD MetadataColor AS JSON_VALUE(Metadata, '$.color');
10 GO
11
12 -- Create index on the computed column
13 CREATE NONCLUSTERED INDEX IX_Product_Metadata_Color
14 ON Product (MetadataColor);
15 GO
16
17 -- Update products with metadata
18 UPDATE Product SET Metadata = N'{"color":"blue","size":"large","material":"cotton"}'
19 WHERE ProductID = 1;
20
21 UPDATE Product SET Metadata = N'{"color":"red","size":"small","material":"silk"}'
22 WHERE ProductID = 2;
23 GO

```

(1 fila afectada)

(1 fila afectada)

Hora de finalización: 2026-09-21T09:52:24.8067244+02:00

```

1  USE EcommerceDB;
2  GO
3
4  -- Query JSON data
5  SELECT
6      ProductID,
7      ProductName,
8      JSON_VALUE(Metadata, '$.color') AS Color,
9      JSON_VALUE(Metadata, '$.size') AS Size,
10     JSON_VALUE(Metadata, '$.material') AS Material
11 FROM Product
12 WHERE JSON_VALUE(Metadata, '$.color') = 'blue';

```

Resultados		Mensajes			
	ProductID	ProductName	Color	Size	Material
1	1	Wireless Mouse	blue	large	cotton

Create a partitioned order table

```

1  USE EcommerceDB;
2  GO
3
4  -- Create partition function for order dates
5  -- Use RANGE RIGHT for date columns to keep same-day values together
6  CREATE PARTITION FUNCTION PF_OrderDate (DATE)
7  AS RANGE RIGHT FOR VALUES
8  ('2025-01-01', '2025-04-01', '2025-07-01', '2025-10-01');
9
10 -- Create partition scheme (single filegroup recommended)
11 CREATE PARTITION SCHEME PS_OrderDate
12 AS PARTITION PF_OrderDate ALL TO ([PRIMARY]);
13
14 -- Create partitioned Order table
15 -- Include OrderDate in primary key for clustered index alignment
16 CREATE TABLE [Order] (
17     OrderID BIGINT IDENTITY(1,1),
18     OrderDate DATE NOT NULL,
19     CustomerName NVARCHAR(100) NOT NULL,
20     TotalAmount DECIMAL(12,2) NOT NULL,
21     OrderStatus NVARCHAR(20) DEFAULT 'Pending',
22     CONSTRAINT PK_Order PRIMARY KEY (OrderID, OrderDate),
23     CHECK (TotalAmount > 0),
24     CHECK (OrderStatus IN ('Pending', 'Processing', 'Shipped', 'Delivered', 'Cancelled'))
25 ) ON PS_OrderDate(OrderDate);
26
27 -- Create partitioned index
28 CREATE NONCLUSTERED INDEX IX_Order_Customer
29 ON [Order](CustomerName)
30 ON PS_OrderDate(OrderDate);
31 GO
32
33 -- Insert sample orders
34 INSERT INTO [Order] (OrderDate, CustomerName, TotalAmount, OrderStatus) VALUES
35 ('2025-01-15', 'John Smith', 299.97, 'Delivered'),
36 ('2025-02-20', 'Jane Doe', 149.99, 'Shipped'),
37 ('2025-06-10', 'Bob Johnson', 449.95, 'Processing');
38 GO

```

Mensajes

Se ha creado correctamente el esquema de partición 'PS_OrderDate'.

(3 filas afectadas)

Hora de finalización: 2026-09-21T09:57:01.5006267+02:00

```

1  USE EcommerceDB;
2  GO
3
4  -- Query by partition
5  SELECT
6      $PARTITION.PF_OrderDate(OrderDate) AS PartitionNumber,
7      COUNT(*) AS OrdersInPartition,
8      MIN(OrderDate) AS MinDate,
9      MAX(OrderDate) AS MaxDate
10 FROM [Order]
11 GROUP BY $PARTITION.PF_OrderDate(OrderDate);

```

	PartitionNumber	OrdersInPartition	MinDate	MaxDate
1	2	2	2025-01-15	2025-02-20
2	3	1	2025-06-10	2025-06-10

Create order details with SEQUENCE

```

1  USE EcommerceDB;
2  GO
3
4  -- Create SEQUENCE for order line items
5  CREATE SEQUENCE OrderLineSequence
6      START WITH 1
7      INCREMENT BY 1;
8
9  -- Create OrderDetail table
10 CREATE TABLE OrderDetail (
11     OrderLineID INT PRIMARY KEY,
12     OrderID BIGINT NOT NULL,
13     OrderDate DATE NOT NULL,
14     ProductID INT NOT NULL,
15     Quantity INT NOT NULL,
16     UnitPrice DECIMAL(10,2) NOT NULL,
17     LineTotal AS (Quantity * UnitPrice),
18     CHECK (Quantity > 0),
19     CHECK (UnitPrice > 0),
20     FOREIGN KEY (OrderID, OrderDate) REFERENCES [Order](OrderID, OrderDate),
21     FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
22 );
23 GO
24
25 -- Insert order details using SEQUENCE
26 INSERT INTO OrderDetail (OrderLineID, OrderID, OrderDate, ProductID, Quantity, UnitPrice)
27 VALUES
28     (NEXT VALUE FOR OrderLineSequence, 1, '2025-01-15', 1, 2, 99.99),
29     (NEXT VALUE FOR OrderLineSequence, 1, '2025-01-15', 2, 1, 149.99),
30     (NEXT VALUE FOR OrderLineSequence, 2, '2025-02-20', 1, 3, 99.99);

```

Mensajes

(3 filas afectadas)

Hora de finalización: 2026-09-21T10:11:22.7033073+02:00

```

1  USE EcommerceDB;
2  GO
3
4  SELECT * FROM OrderDetail;

```

	OrderLineID	OrderID	OrderDate	ProductID	Quantity	UnitPrice	LineTotal
1	1	1	2025-01-15	1	2	99.99	199.98
2	2	1	2025-01-15	2	1	149.99	149.99
3	3	2	2025-02-20	1	3	99.99	299.97

Verify database objects

```

1  USE EcommerceDB;
2  GO
3
4  -- Verify constraints work
5  -- This should fail: negative price
6  INSERT INTO Product (ProductName, CategoryID, SupplierID, BasePrice, StockQuantity)
7  VALUES ('Invalid', 1, 1, -50, 10);
8

```

% 6 0 ↑ ↓ Línea: 9, Carácter: 2 TABULACIONES CRLF

Mensajes

Mens. 547, Nivel 16, Estado 0, Línea 6
Instrucción INSERT en conflicto con la restricción CHECK 'CK_Product_BasePri_534D60F1'. El conflicto
Se terminó la instrucción.

```

1  USE EcommerceDB;
2  GO
3
4  -- Verify JSON queries work
5  SELECT ProductName, JSON_VALUE(Metadata, '$.color') AS Color
6  FROM Product
7  WHERE Metadata IS NOT NULL;
8
9  -- Verify partitioning
10 SELECT $PARTITION.PF_OrderDate(OrderDate) AS Partition, COUNT(*) AS RecordCount
11 FROM [Order]
12 GROUP BY $PARTITION.PF_OrderDate(OrderDate);
13
14 -- Verify temporal table
15 SELECT ProductID, CurrentPrice, SysStartTime, SysEndTime
16 FROM ProductPrice FOR SYSTEM_TIME ALL
17 ORDER BY ProductID, SysStartTime;

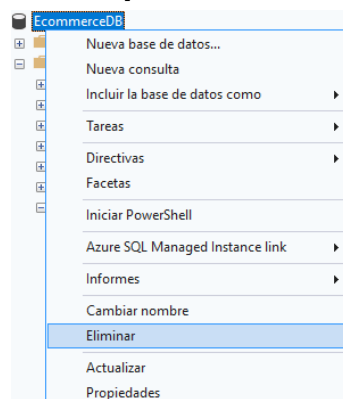
```

ProductID	CurrentPrice	SysStartTime	SysEndTime
1	99.99	2026-09-21 07:47:54.7781940	2026-09-21 07:47:54.7791956
2	109.99	2026-09-21 07:47:54.7791956	9999-12-31 23:59:59.9999999
3	149.99	2026-09-21 07:47:54.7781940	9999-12-31 23:59:59.9999999

Partition	RecordCount
1	2
2	1

ProductName	Color
1	Wireless Mouse
2	Cotton T-Shirt

Cleanup



Enlace al repositorio de GitHub

[GitHub Repository](#)